

DJANGO MODELS

The Definitive Source of Truth for Your Data

DATABASE ARCHITECTURE & ORM FUNDAMENTALS



Fundamental Architecture

Defining structures that bridge Python and SQL

| ANATOMY OF A MODEL

Core Principles

-  Subclasses `django.db.models.Model`
-  Each attribute maps to a database column
-  Auto-generated Database-Access API

```
from django.db import models

class Person(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
```

Django translates this Python class into a CREATE TABLE statement tailored for your backend.

| THE ROLE OF FIELD CLASSES



Column Types

Determines SQL storage (e.g., INTEGER, VARCHAR, TEXT) based on the specific field instance.



Form Widgets

Sets default HTML rendering (e.g., or) for Django-generated forms.



Validation

Enforces minimal requirements for clean data in both admin interfaces and end-user forms.

ESSENTIAL FIELD OPTIONS

Option	Default	Functionality
null	False	Stores empty values as NULL in the database.
blank	False	Validation-related; allows empty values in forms.
choices	None	Sequence of tuples to use as choices for the field.
default	None	The default value for the field (value or callable).
primary_key	False	Marks the field as the table's primary key.

| DATA RELATIONSHIPS

Many-to-one: Defined using `ForeignKey`. Requires a positional argument for the related class.

Many-to-many: Defined using `ManyToManyField`. Recommended to be plural (e.g., toppings).

One-to-one: Defined using `OneToOneField`. Ideal for extending existing objects (inheritance).

| INTERMEDIATE M2M TABLES

Extra Fields on Relations

When associating data with the relationship itself (e.g., *date joined* in a group), use the `through` argument.

```
class Membership(models.Model):  
    person = models.ForeignKey(Person, ...)  
    group = models.ForeignKey(Group, ...)  
    date_joined = models.DateField()
```

This allows precise tracking of metadata for complex relational connections.



| QUERY EFFICIENCY

1

SINGLE SOURCE OF TRUTH

The Power of ORM

Django Models abstract away SQL complexity, allowing developers to focus on business logic while maintaining strict data integrity across the entire application stack.

$$T_{\text{complexity}} = \frac{\sum (\text{Fields} + \text{Methods})}{\text{Models}}$$

| INHERITANCE PATTERNS

 Hierarchy Diagram

Abstract Base

Common info for children; no database table created for parent.



Multi-Table

Each model has its own table; parent-child link via OneToOne.



Proxy Models

Changes Python behavior only; no new database table created.

| PROJECT SCALE & MIGRATIONS



Start Phase Expansion Enterprise Scale

As complexity grows, manage.py makemigrations becomes critical for managing schema evolution safely.

| BEST PRACTICES



Logical Organization

-  Use model packages for large apps instead of a single `models.py`.
-  Define `__str__()` for useful admin display.
-  Use `get_absolute_url()` for unique object identity.
-  Always call `super().save()` when overriding.

Questions?

Deep Dive into Django 6.0 Models

Explore the Full Documentation

`docs.djangoproject.com/en/6.0/topics/db/models/`